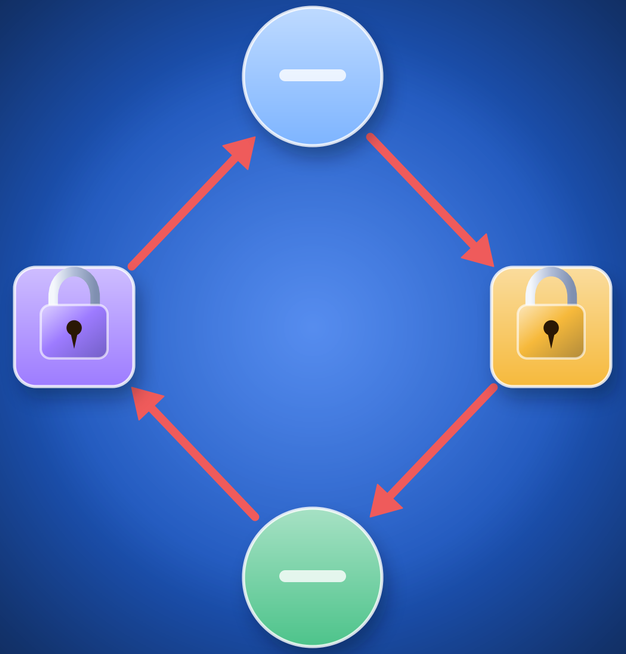
**Institute of Technology of Cambodia**
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 9

# Deadlock, Lock Order and Recovery

Lab 9 guide: make two workers wait for each other, draw the cycle, and remove it with one lock order.



Heng Rathpisey · Department GIC · Academic year 2025–2026

# Learning objectives

1. Draw holdings and requests that form a circular wait.
2. Reproduce a two-worker conflict with bounded waits, and read how a timeout recovers from it.
3. Apply one global lock order, and explain why it removes the cycle.

THIS LAB

**Lab 9**

First the concepts, with pictures. Then the lab on the server. Wait traces and the site-to-site exercise are optional extensions, so they come last.

DISCUSSION

# Two people each hold one key and want the other's. What happens?

Neither will give theirs up first. How long will they wait? How could they have avoided it?

CONCEPTS FIRST

# Holding, waiting and cycles

Three ideas: a worker can hold one resource while it asks for another, waits can form a circle, and a rule or a timeout can break it.

# Four conditions for a deadlock

01

**Mutual exclusion**

A resource can be used by only one worker at a time.

02

**Hold and wait**

A worker holds one resource while it asks for another.

03

**No preemption**

A resource cannot be taken away. Its holder must release it.

04

**Circular wait**

Worker A waits for B, and B waits for A: the waits form a circle.

All four must hold together. Removing any one of them prevents a deadlock. This lab removes the fourth.

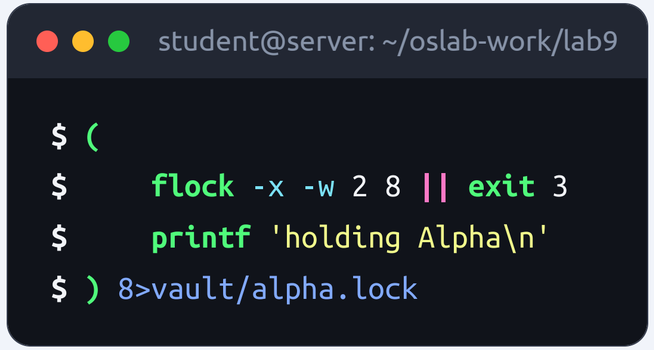
# Draw it as holdings and requests

| Worker | Holds | Requests |
|---|---|---|
| **A** | Alpha | Beta |
| **B** | Beta | Alpha |

Draw an arrow from a resource to the worker that holds it, and from a worker to the resource it waits for.

Here: Alpha to A, A to Beta, Beta to B, B to Alpha. The arrows close a circle, so neither worker can go on.

# A lock with a time limit




A data file holds the data. A lock file stands for cooperative exclusive access. Creating a file named locked is not a lock.

flock -x -w 2 waits at most two seconds for the lock. If it does not get it, the status is not zero, and the script can stop waiting.

Descriptor 8 stays open until the group of commands ends, and that is when the lock is released.

# Two ways to deal with a deadlock

BEFORE IT HAPPENS

## Prevention: a global lock order

Every worker takes Alpha before Beta. A worker that waits for Alpha holds nothing yet, so no circle can form.

AFTER IT HAPPENS

## Recovery: a timeout

A worker that waits too long gives up and releases what it holds. The others can then go on.

IN PRACTICE

## Use both

Ordering stops this circle. Timeouts also bound waits that come from other causes. Neither gives fairness.

LAB 9

# The vault that freezes

QuantumTech's Alpha and Beta recovery vaults freeze during a drill. Model them, watch the wait, and repair the order.

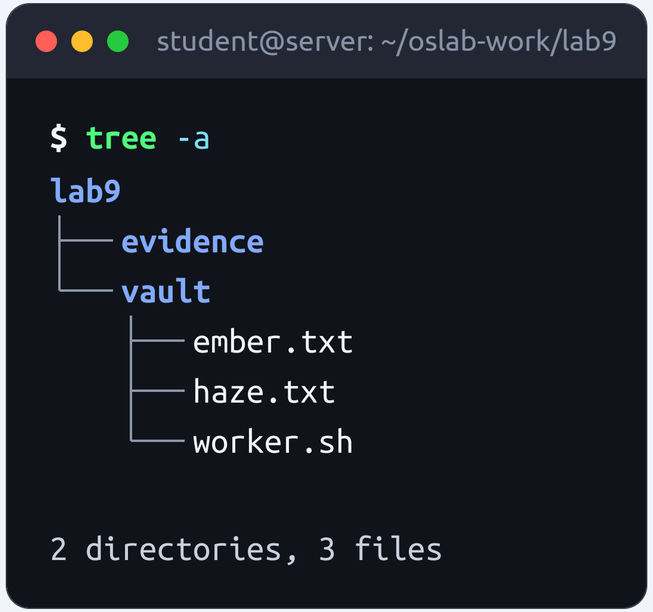
# The scenario

SCENARIO

## QuantumTech's recovery vaults

Alpha and Beta freeze during a synchronization drill. Model them with two lock files you own, observe a bounded wait cycle, and repair the acquisition order.

Your two lock names and the worker who starts first are your own. This guide shows example values.



# Inside the 120 minutes

| Minutes | What happens |
|---|---|
| **0 to 10** | Setup |
| **10 to 25** | Guided: locks and bounded waiting |
| **25 to 35** | Prediction |
| **35 to 55** | Core 1: reproduce the deadlock |
| **55 to 70** | Core 2: repair it with a global order |

| Minutes | What happens |
|---|---|
| **70 to 85** | Instructor demonstration, Plus and Challenge |
| **85 to 100** | Live checkpoint |
| **100 to 110** | Debrief |
| **110 to 120** | Submit |

# Six helper commands, and why you use them

| Command | What it does |
|---|---|
| `oslab values lab9` | Shows your two lock names and which one worker A takes first |
| `oslab predict lab9` | Saves your prediction once, before the experiment |
| `oslab hint lab9 1` | A hint, levels 1 to 3. Hints cost no points |
| `oslab check lab9` | Runs your worker.sh in both modes and shows your milestones. It takes about 10 seconds |
| `oslab checkpoint lab9` | Opens when the instructor says so, near the end |
| `oslab debrief lab9` | After the lab: you give the right answers to your prediction questions |

Before the lab, run oslab doctor and oslab prelab lab9. It takes about ten minutes.

# How you work

## Core, Plus, Challenge

Core is for everyone. Plus and Challenge are for students who finish early. Full marks need only Core and the checkpoint.

## AI tools

Allowed in Core, Plus and Challenge, so test every suggestion. Not allowed in the prediction and the checkpoint.

## Every wait has a limit

You only ever start your own two workers, always with timeout. Use the saved PIDs, and never use pkill.

In Core 2 you decide the global lock order yourself. You write the rule, and you explain why it removes the cycle.

# Setup: start the lab

student@server: ~

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab9
$ cd "$OSLAB_WORKSPACE/lab9"
$ mkdir -p evidence
$ oslab values lab9
```

- The first line sets OSLAB_WORKSPACE to ~/oslab-work, unless it is already set. Type it as it is.
- oslab start lab9 makes the lab folder.
- cd takes you into it, and mkdir -p makes the evidence folder.
- oslab values lab9 prints your own lock names.

# Your values: two lock names and a start

## The last command prints them

example: your values will be different

```
$ oslab values lab9
Your personal values for lab9 (your-account):
  res1     = haze
  res2     = ember
  a_first  = haze
```

## What they mean

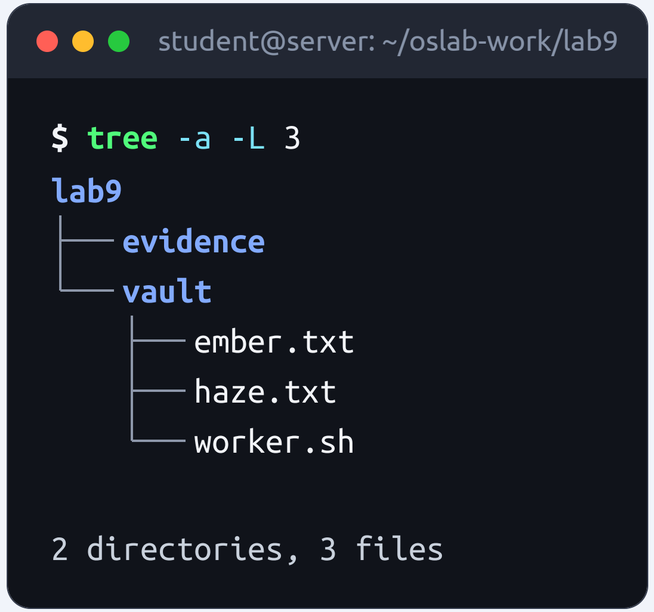
- res1 and res2 are the names of your two locks.
- a_first is the lock that worker A takes first. Worker B always takes them in the other order.
- Your own names are different. Write them down.

# The capital words are placeholders

| The instruction says | With the example values, you type |
|---|---|
| **`vault/RES1.lock`** | `vault/haze.lock` |
| **`cat vault/RES1.txt`** | `cat vault/haze.txt` |

The instruction cannot know your names, so it writes RES1 and RES2 where your values belong. Never type the capital words. Type your own names.

# Your folder




vault/ holds the data of your two resources, named with your own lock names, and worker.sh, with one TODO for you.

oslab check lab9 starts with the first line passing and the others saying TRY. The hidden .oslab-managed.json is the helper's. Leave it.

GUIDED

# Locks and bounded waiting

Ten to twenty-five minutes, with the instructor. See what a lock is, and how a worker can stop waiting for one.

# Steps 1 and 2: look, then take one lock

example names: use your own

```
$ cat vault/haze.txt vault/ember.txt
$ ls vault
$ (
$     flock -x -w 2 8 || exit 3
$     printf 'holding haze\n'
$ ) 8>vault/haze.lock
```

The text files hold data. A lock file is separate: flock makes workers take turns on it. Creating a file called locked is not a lock.

File descriptor 8 stays open until the subshell ends, and the lock is held as long as it is open. -w 2 means wait at most 2 seconds.

# Step 3: one lock, two workers

example name: use your own

```
$ (
$     flock -x -w 2 8 || exit 3
$     sleep 3
$ ) 8>vault/haze.lock &
$ holder=$!
$ sleep 0.2
$ flock -x -w 1 vault/haze.lock true
$ printf 'contender status=%s\n' "$?"
$ wait "$holder"
```

One subshell holds the lock for 3 seconds. The second command waits only 1 second, so its status is not 0: the wait timed out.

THINK

## Observe

What is held while another worker waits? Does a timeout prevent a deadlock, or only let a worker stop waiting?

PREDICT

# Guess first, then try

Before you start any worker. Ten minutes.

# The prediction

```
student@server: ~/oslab-work/lab9

$ oslab predict lab9
```

It asks four questions and one sentence about your own two locks. Draw the two workers and the two locks on paper first. Answer alone, without AI.

PREDICTION

## The rules

The first answer is the one that counts. A wrong guess costs nothing. You compare it with the real result in the debrief.

# Reproduce the deadlock

Twenty minutes, from minute 35 to 55. Make worker A and worker B wait for each other, and read what the timeouts say.

# Step 1: read the worker, part 1

vault/worker.sh (1 of 3)

```
#!/usr/bin/env bash
set -euo pipefail
# (the check of the two arguments is left out here)
role=$1
mode=$2
vault=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)
res1=haze
res2=ember
mkdir -p "$vault/coord"
# Worker A takes haze first and ember second. Worker B the other way.
first=$res1; second=$res2
# TODO: in ordered mode, every worker must use one global order.
if [[ "$role" = B ]]; then tmp=$first; first=$second; second=$tmp; fi
```

Read it with cat -n vault/worker.sh. The values are an example: yours hold your own names, and your own first lock for A.

set -euo pipefail stops on an error, on an unset variable, and when any command in a pipeline fails.

The vault= line finds the script's own folder. B reverses the order by default: the TODO is yours to fix in Core 2.

# Step 1: the first lock and the barrier, part 2

vault/worker.sh (2 of 3)

```
exec 8>"$vault/$first.lock"
flock -x -w 2 8 || { echo "$role first lock timeout"; exit 3; }
printf '%s holds %s\n' "$role" "$first"
sleep "${HOLD:-0}"   # teaching delay; keep this line
if [[ "$mode" = opposite ]]; then
    touch "$vault/coord/$role.ready"
    other=B
    [[ "$role" = B ]] && other=A
    for step in {1..30}; do
        [[ -f "$vault/coord/$other.ready" ]] && break
        sleep 0.1
    done
    [[ -f "$vault/coord/$other.ready" ]] || { echo 'barrier timeout'; exit 4; }
fi
```

Each worker takes its first lock on descriptor 8, waiting two seconds at most. The sleep line is a teaching delay: keep it.

The barrier makes both workers announce their first lock before either asks for the second. It is used only in opposite mode.

# Step 1: the second lock, part 3

vault/worker.sh (3 of 3)

```
exec 9>"$vault/$second.lock"
printf '%s requests %s\n' "$role" "$second"
flock -x -w 2 9 || { echo "$role second lock timeout"; exit 5; }
printf '%s holds both\n' "$role"
```

After the barrier, each worker asks for its second lock, on descriptor 9, and reports what it holds.

SYNTAX

### Why exec 8> and exec 9>?

exec with only a redirection opens the file for the rest of the script, so the lock lasts until the script ends.

# Step 2: two workers, opposite order

student@server: ~/oslab-work/lab9

```
$ rm -f -- vault/coord/A.ready vault/coord/B.ready
$ timeout 6 bash vault/worker.sh A opposite > evidence/opposite-a.txt 2>&1 &
$ worker_a=$!
$ timeout 6 bash vault/worker.sh B opposite > evidence/opposite-b.txt 2>&1 &
$ worker_b=$!
$ if wait "$worker_a"; then rc_a=0; else rc_a=$?; fi
$ if wait "$worker_b"; then rc_b=0; else rc_b=$?; fi
$ {
$   printf 'opposite A=%s B=%s\n' "$rc_a" "$rc_b"
$   cat evidence/opposite-a.txt evidence/opposite-b.txt
$ } > evidence/opposite.txt
$ cat evidence/opposite.txt
```

Clear only the two old ready markers, then start exactly two workers, each with timeout 6.

The if wait ... construct keeps the status even when it is not zero. A status of 124 means timeout itself stopped the worker.

# Step 3: draw it, and explain the recovery

## What to do

1. Draw what each worker holds and requests at the barrier.
2. At least one worker ends with status 5, the timeout of the second lock.
3. When it exits, its first lock is released, so the other worker may finish instead of timing out too.
4. Explain that recovery in your report. Do not claim that both workers must fail.

STUCK?

### Hints

oslab hint lab9 1, then 2, then 3. AI is allowed here: test every suggestion.

# Repair it with a global order

Fifteen minutes, from minute 55 to 70. One rule for both workers, so that no worker waits for a lock while holding the one the other needs.

# Step 1: one order for everyone

## Edit the TODO

- Open the worker with nano vault/worker.sh and find the TODO.
- In ordered mode, both workers must take the locks in the same order.
- In opposite mode, worker B must still take them the other way round.
- Keep the barrier only for opposite mode. Write your rule with $first and $second, not with your lock names.

YOUR TURN

### You write it

Nobody gives you the lines. Ask: which line decides first and second, and for which workers and modes?

# Step 2: run the ordered case

student@server: ~/oslab-work/lab9

```
$ timeout 6 bash vault/worker.sh A ordered > evidence/ordered-a.txt 2>&1 &
$ worker_a=$!
$ timeout 6 bash vault/worker.sh B ordered > evidence/ordered-b.txt 2>&1 &
$ worker_b=$!
$ if wait "$worker_a"; then rc_a=0; else rc_a=$?; fi
$ if wait "$worker_b"; then rc_b=0; else rc_b=$?; fi
$ {
$   printf 'ordered A=%s B=%s\n' "$rc_a" "$rc_b"
$   cat evidence/ordered-a.txt evidence/ordered-b.txt
$ } > evidence/ordered.txt
$ cat evidence/ordered.txt
```

Both statuses should be 0.

**WHY**

### Think

Why can a worker that waits for the first lock not hold the second lock at the same time?

# Step 3: clean up

example names: use your own

```
$ jobs
$ flock -n vault/haze.lock true; printf 'haze free: status=%s\n' "$?"
$ flock -n vault/ember.lock true; printf 'ember free: status=%s\n' "$?"
```

No worker may still run, and both locks must be free. Both statuses should be 0.

The timeouts stay in the script. Ordering prevents this circular wait, but timeouts still bound other delays. Ordering does not give fairness, and does not make every larger system safe.

# Step 4: check your work

The four Core milestones must show PASS: the opposite conflict, ordered mode, a rule that does not depend on names, and the evidence files.

student@server: ~/oslab-work/lab9

```
$ oslab check lab9
```

CHECK

## What the check does

It runs your worker.sh in a disposable folder: in opposite mode, in ordered mode, and in ordered mode again with two other lock names. The last test fails if your rule only works for your own names.

# The three runs

| Run | Intended evidence |
| --- | --- |
| **Opposite, two workers** | Both hold distinct first locks. The second lock conflicts, and a timeout recovers |
| **Ordered, two workers** | Both complete, one after the other, through the first lock of the order |
| **Ordered, two other lock names** | The rule still works, so it is written with $first and $second |

Status 124 means the outer timeout fired. Do not call an unexplained timeout a recovery. Repeat the opposite case only after you delete the two ready markers.

# If something does not work

## Old ready markers

Reusing them makes the barrier pass when it should not. Clear only A.ready and B.ready.

## Lock files stay

Never delete an active lock file. Its existence is normal.

## No pkill

Use the saved PIDs and the bounded waits. See man flock and man timeout.

# Plus: slow the first lock down

student@server: ~/oslab-work/lab9

```
$ HOLD=1 timeout 6 bash vault/worker.sh A ordered &
$ HOLD=1 timeout 6 bash vault/worker.sh B ordered &
$ wait
```

HOLD=1 makes each worker keep its first lock for one second before it asks for the second. Run both workers in ordered mode with it.

PLUS

**Answer in evidence/plus.txt**

Do both workers still finish? One of them waits longer. What does it wait for?

First watch the instructor draw the two workers and the two locks and show the circle on the board. Plus and Challenge are not needed for full marks.

# Challenge: three workers, three locks

CHALLENGE

## The circle

Three locks x, y and z. A holds x and requests y. B holds y and requests z. C holds z and requests x.

YOUR TURN

## Answer in evidence/plus.txt

Draw the circle. Choose a global order, and say which worker must change the order of its two locks.

LIVE CHECKPOINT

# Show what you can do alone

Fifteen minutes near the end of the lab. Worth 3 of the 10 points.

# Why is there a checkpoint?

### It shows what you can do alone

For most of the lab you may use hints and AI tools. The checkpoint is the one moment without them.

### The names are new

The case uses new lock names, made for you at that moment. An answer copied from a neighbour does not fit.

### The instructor sees who needs help

Your answers arrive at once, so a common mistake can be explained before the next lab.

### It is fair

Everyone gets the same kind of task, with their own names.

# What happens, step by step

**1. The instructor opens it**
Close your AI tools

→

**2. oslab checkpoint lab9**
Read the case, answer four questions

→

**3. The file task**
Copy your worker, new names

→

**4. oslab check lab9**
A fifth milestone must pass

## Before it opens

The command says that the checkpoint is not open yet. Wait for the instructor.

## While you work

Use your terminal and what you learned in this lab. No AI tools, and no help from a neighbour.

## When you finish

oslab check lab9 shows whether the fifth milestone, checkpoint, passes. You can fix your copy and check again.

# What the checkpoint asks

| Part | What you do |
|---|---|
| **Four questions** | Read the case with a third worker, made for you at this moment. Answer them in the terminal. |
| **The file task** | Copy your worker script, change only the two res1= and res2= lines to the new names, run the ordered mode, and save the logs in checkpoint.txt. |
| **The check** | Run oslab check lab9. The fifth milestone, checkpoint, must show PASS. |

This is why Core 2 asked for a rule written with $first and $second, not with your own lock names.

DEBRIEF AND SUBMIT

# Look back, then hand in

Ten minutes of debrief, ten to submit.

# Debrief: look at your prediction again

student@server: ~/oslab-work/lab9

```
$ cat "$OSLAB_WORKSPACE/.records/lab9-predict.json"
$ oslab debrief lab9
```

oslab saved your prediction in a hidden folder, .records. It stays as it was, so the comparison shows which answers you confirmed and which you corrected.

DISCUSS

### Class discussion

Your timeout rescued the opposite run. Why is wait less not a repair for a deadlock?

# Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab9/evidence"
$ cd "$OSLAB_WORKSPACE/lab9"
$ cp -- vault/worker.sh "$SUBMISSION_REPO/lab9/"
$ cp -- evidence/opposite.txt evidence/ordered.txt "$SUBMISSION_REPO/lab9/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

If you did Plus or Challenge, copy evidence/plus.txt the same way. The course website checks that these exact names are there. Leave no worker running.

# Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab9
├── evidence
│   ├── opposite.txt
│   └── ordered.txt
├── README.md
└── worker.sh

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

# Submit 3: commit and push

student@server: ~

```
$ cd ~/os-gic-YOUR_ID
$ git pull
$ git add os-lab-YOUR_ID/lab9
$ git commit -m "Lab 9"
$ git push
```

git pull comes first, in case you pushed from your own computer since.

Replace YOUR_ID with your own ID, as before.

# How it is marked: 10 points

| Evidence for the lab objectives | Points |
|---|---|
| **Worker: oslab check lab9 passes the four Core milestones (opposite conflict, ordered mode, rule independent of names, evidence files)** | 2 |
| **Tests: opposite.txt and ordered.txt show the conflict and the repair, and the README explains the recovery by timeout** | 2 |
| **Prediction saved in time, and corrected with oslab debrief lab9** | 2 |
| **Live checkpoint: the answers and your worker running with the new names** | 3 |
| **Complete submission: the required files in your repository, on time** | 1 |

Every row is marked from your oslab records and the files in your repository, so run oslab check lab9 until it shows what you expect. Your prediction earns its first point for being made, not for being right. Plus and Challenge are not needed for full marks.

# Common mistakes

| Mistake | What you see | Fix |
|---|---|---|
| **Claiming both workers must time out** | One finishes after the other gives up | Explain the recovery event |
| **The barrier left in ordered mode** | A worker waits for a partner that cannot arrive | Skip the barrier when ordered |
| **Removing the timeouts after ordering** | A future wait with no limit | Keep every flock bounded |
| **Reusing the ready markers** | A false coordination condition | rm only A.ready and B.ready first |
| **A rule written with your own lock names** | The last check, with other names, fails | Write it with $first and $second |

BEYOND THE CORE

# Wait traces and site-to-site sync

Optional extensions. They stay bounded, and they use only a narrow folder the instructor prepared.

# Looking at who waits

student@server: ~/oslab-work/lab9

```
$ timeout 6 bash vault/worker.sh A opposite &
$ ps -o pid,ppid,stat,wchan:20,comm -p $!
$ wait
```

While a worker waits for a lock, ps shows it asleep, state S. The wchan column names the kernel place where it sleeps.

Take one sample, and say what you saw. A late sample may miss the wait entirely.

# Site-to-site synchronization

CONSENT

### Consent first

Two people agree on a narrow folder the instructor prepared. Nobody writes into another person's home.

ORDER

### Same order on both sides

Both sites must take their locks in the same global order, or the circle comes back across machines.

SAFE

### Bounded and logged

Every wait has a limit, and each side keeps a log of what it held and asked for.

# Key takeaways

01

**Four conditions**

Mutual exclusion, hold and wait, no preemption and circular wait. Break one.

02

**Draw the edges**

Holdings and requests that close a circle are a deadlock.

03

**Order prevents, timeout recovers**

One global lock order stops this circle. A timeout only ends a wait that has started.

04

**Keep every wait bounded**

Even with an order, a limit stops a hidden problem from hanging for ever.

# Next session

COMING UP

## Lab 10: backups, archives and cron

Make a backup you can restore, and schedule it to run by itself.

SOURCE

## Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 8, Deadlocks.